

C Systems Lab

A from-scratch systems programming roadmap in C

Purpose

This roadmap is designed to be followed after completing basic C programming exercises. The goal is not to collect toy programs, but to progressively understand memory, operating-system interfaces, processes, concurrency, networking, storage, and infrastructure by building working systems yourself.

Rule: Do not rush to the next project. Each project should leave you with a working implementation, tests, documentation, and a clear explanation of what happens underneath.

Phase 1 — Memory & Core Data Structures

#	System to build	Main concepts
01	Dynamic Array	malloc, realloc, free, capacity vs length, pointer arithmetic, ownership
02	C String Library	strlen, strcpy, strcmp, memcpy, memset, strchr; bounds and null termination
03	Generic Vector	void*, element size, generic storage, callbacks, memory ownership
04	Linked List	singly/doubly linked lists, insertion/deletion, lifetime management
05	Hash Table	hashing, buckets, collisions, resizing, load factor

Phase 2 — Unix/Linux Utilities

#	System to build	Main concepts
06	ccat	open, read, write, close, file descriptors, error handling
07	cwc	buffered file processing, lines, words, bytes, command-line arguments
08	cgrep	pattern matching, files, directories, recursion, flags
09	Mini File Utility	stat, directory traversal, permissions, metadata

Phase 3 — Processes & the Operating System

#	System to build	Main concepts
10	Mini Shell	fork, exec, wait, environment, exit status, redirection
11	Pipes & Job Control	pipe, dup2, pipelines, background jobs, process groups
12	Process Monitor	PIDs, /proc, process state, CPU/memory information
13	Thread Pool	pthread, mutexes, condition variables, work queues, synchronization
14	Producer–Consumer System	bounded queues, blocking, race conditions, synchronization

Phase 4 — Networking

#	System to build	Main concepts
15	TCP Echo Server	socket, bind, listen, accept, recv, send, TCP byte streams
16	TCP Chat Server	multiple clients, connection state, broadcast, concurrency
17	HTTP Server	HTTP parsing, request/response, headers, status codes, static files
18	Concurrent HTTP Server	thread-per-connection, thread pool, load testing
19	Event-Driven HTTP Server	select/poll, then epoll; non-blocking sockets and event loops
20	Reverse Proxy	upstream connections, forwarding, timeouts, load balancing, health checks

Phase 5 — Storage & Database Internals

#	System to build	Main concepts
21	Key-Value Store	GET/SET/DELETE, hash table, command protocol, persistence
22	Persistent Log Store	append-only log, record format, recovery, corruption handling
23	Indexing & Compaction	indexes, tombstones, compaction, read/write trade-offs
24	Mini Database	parser, execution layer, storage layer, basic transactions/WAL

Phase 6 — Infrastructure / Linux Internals

#	System to build	Main concepts
25	Mini Container	namespaces, chroot/pivot_root concepts, mounts, process isolation
26	Resource Limiter	cgroups concepts, CPU/memory limits, process accounting
27	Service Supervisor	start/stop/restart, child processes, signals, logs, crash recovery
28	Mini Load Balancer	health checks, round robin, connection handling, failure behavior

How to Build Each Project

- 1. Define the boundary** — Write down exactly what the first version supports. Avoid implementing everything at once.
- 2. Build the smallest working version** — Prefer direct OS/library primitives before adding abstractions.
- 3. Understand the primitives** — For example: understand file descriptors before hiding them behind a wrapper.
- 4. Add failure handling** — Check return values, errno, partial reads/writes, allocation failures, closed connections, and invalid input.
- 5. Test it** — Add deterministic tests and manual integration tests. Include edge cases.
- 6. Measure it** — For relevant systems, record throughput, latency, memory usage, connection counts, or startup time.
- 7. Document the internals** — Explain architecture, important data structures, system calls, ownership, concurrency model, and trade-offs.
- 8. Refactor only after understanding** — Abstraction should follow understanding—not hide the parts you have not learned.

Suggested Repository Layout

```
c-systems-lab/  
  01-memory/  
  02-unix/  
  03-process/  
  04-networking/  
  05-storage/  
  06-infrastructure/  
  benchmarks/  
  docs/  
  tests/  
  Makefile  
  README.md
```

What Every Project Should Contain

- **README.md** — what it does, architecture, build/run instructions, limitations, and what you learned.
- **Source** — clean C with meaningful error handling.
- **Tests** — unit tests where useful plus integration/manual tests.
- **Benchmark** — when performance is relevant.
- **Design notes** — important decisions and rejected alternatives.
- **Known limitations** — explicitly state what the project does not implement.

Recommended Milestone Order

If you want the strongest learning progression, prioritize these milestones: **Dynamic Array** → **Hash Table** → **ccat** → **cgrep** → **Mini Shell** → **Thread Pool** → **TCP Server** → **HTTP Server** → **epoll HTTP Server** → **Reverse Proxy** → **Key-Value Store** → **Mini Container**.

Portfolio / GitHub Strategy

The repository can also generate legitimate profile traffic because every substantial project gives you something technical to publish: architecture diagrams, benchmarks, debugging discoveries, implementation notes, and demos.

Do not optimize the implementation for stars. Optimize for projects that are technically interesting enough that another developer would want to inspect the code.

Definition of Done

A project is complete when you can explain its architecture without reading your own code, reproduce it from a clean checkout, demonstrate its failure cases, explain the relevant OS/network/storage primitives, and document the limitations. A project that merely compiles is **not** finished.